

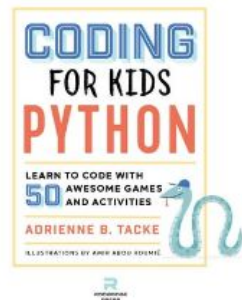


Завдання 1: Чистий Код

Виправляємо хакерський злам та називаємо змінні правильно

✿ Ей, поглянь — одну з програм, яку написав наш друг,
випадково зламали хакери!
Хтось змінив усі назви змінних на ось такий безлад:

```
80 = "Adrienne"  
98_cookie_39 = "Chocolate chip cookies"  
fIrSt_NAME = 20  
LAST_name = "Blue"  
309384 = "Adrienne Tacke"  
Helllloooooooooooooooooo_8392982r = "Software Engineer"  
aPPles_23 = "45"  
_Cittty_43 = "Stockholm"  
0_sTRRRRREEET_34211 = "Horward 45"
```



Джерело: Coding for Kids: Python

Авторка: Адрієнн Такке

| Золоті правила іменування

- 🕸 **Описовість:** Назва має "говорити" про дані всередині.
- >_ **snake_case:** Використовуй малі літери та підкреслення _.
- 1 **Жодних цифр спочатку:** Змінна не може починатися з числа.
- ∞ **Стиль:** Будь послідовним у всьому проекті.



Завдання 2: Менеджер Бази

Створення систем життєзабезпечення станції

Крок 1: Назва станції

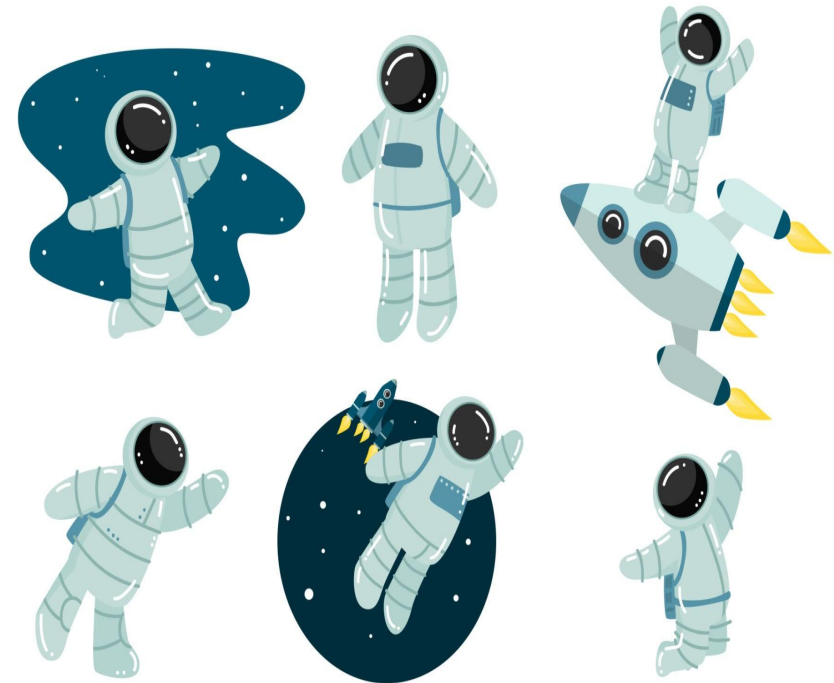
Почнімо з назви нашої бази.

Створіть змінну `station_name` і
присвойте їй будь-яку текстову назву в
лапках.



Крок 2: Скільки людей на борту?

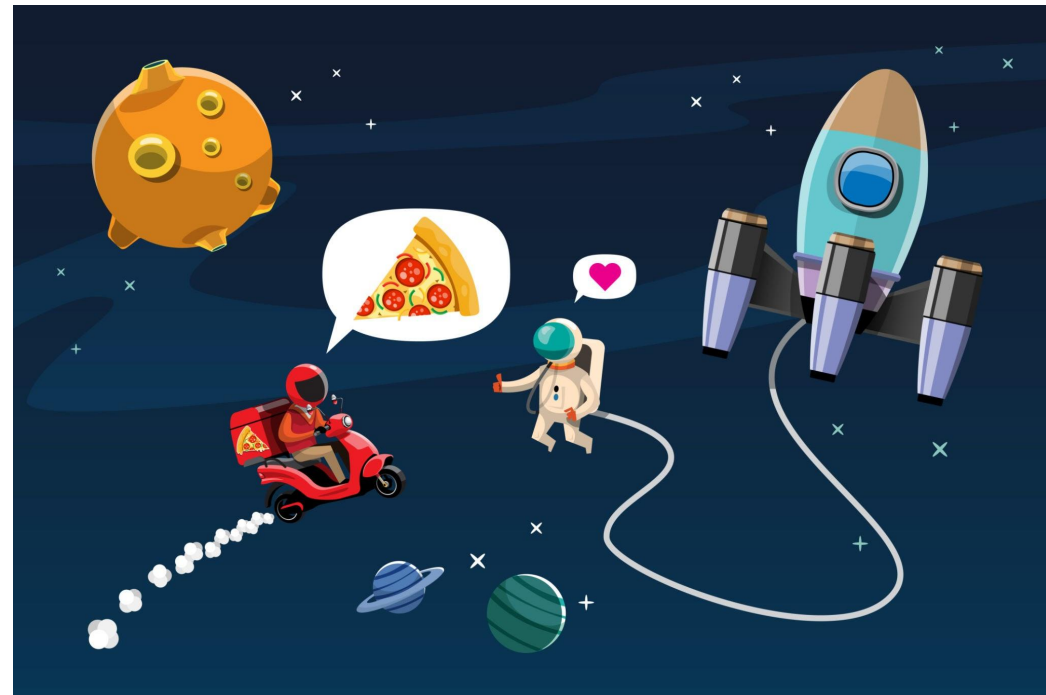
Створіть змінну `crew_members` і запишіть туди ціле число, наприклад, 4.



Крок 3: Контейнери з їжею

Тепер порахуємо запаси.

Створіть змінну `food_containers` і нехай на складі буде 15 коробок.



Крок 4: Порції в коробці

В одній коробці багато їжі. Створіть змінну `portions_per_container` і запишіть туди число 20 (порцій)



Крок 5: Множення запасів *

А тепер перша математика.
Створіть змінну `total_portions`.
Порахуйте загальну кількість порцій їжі
на складі, помноживши кількість
контейнерів на порції в одному
контейнері.



Крок 6: Апетит КОСМОНАВТІВ

Скільки порцій з'їдає ОДИН астронавт за
один день?

Створіть змінну `portions_per_day` і
запишіть туди число 3.



Крок 7: Денна норма всієї станції (*)

Створіть змінну `daily_station_consumption`.
Порахуйте, скільки їжі з'їдає ВСЕ
угруповання за день. Для цього помножте
кількість членів екіпажу на денну порцію
одного астронавта.



Крок 8: Ділення ресурсів (/)

На скільки днів нам вистачить їжі?
Створіть змінну `days_of_food`. Поділіть загальну кількість порцій на складі на денну норму споживання всієї станції.



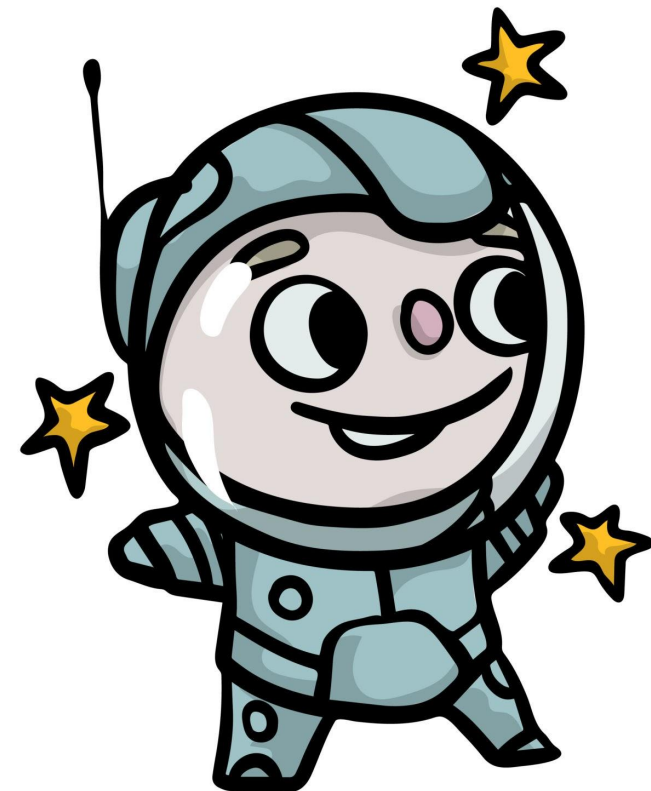
Крок 9: Перший звіт (print)

Перевірмо наші розрахунки. Виведіть на екран змінну `days_of_food` за допомогою команди `print()`. Що у вас вийшло?



Крок 10: Життєзабезпечення

Переходимо до систем безпеки. Створіть логічну змінну `oxygen_system` і присвойте їй значення `True` (система генерації кисню працює).



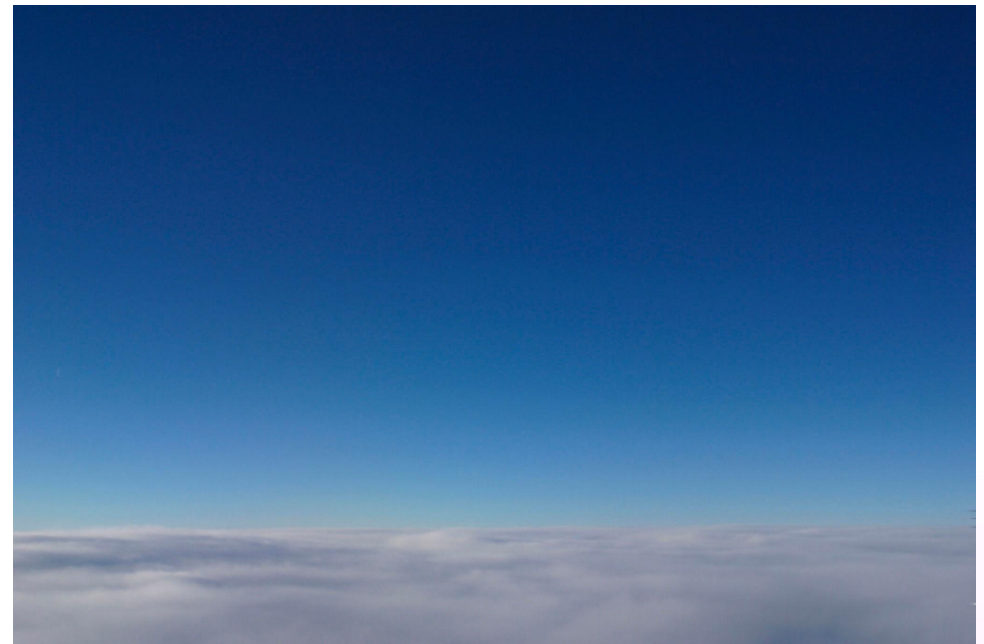
Крок 11: Перевірка пробоїн

Чи є витік повітря через обшивку?
Створіть логічну змінну `air_leak` і
присвойте їй `False` (витоку немає, ми в
безпеці).



Крок 12: Логічне «І» (and)

Створіть змінну `atmosphere_safe`.
Повітря безпечне, якщо система кисню
працює (`oxygen_system`) І (and) витоку
повітря немає (тобто `air_leak == False`).
Запишіть цю умову.



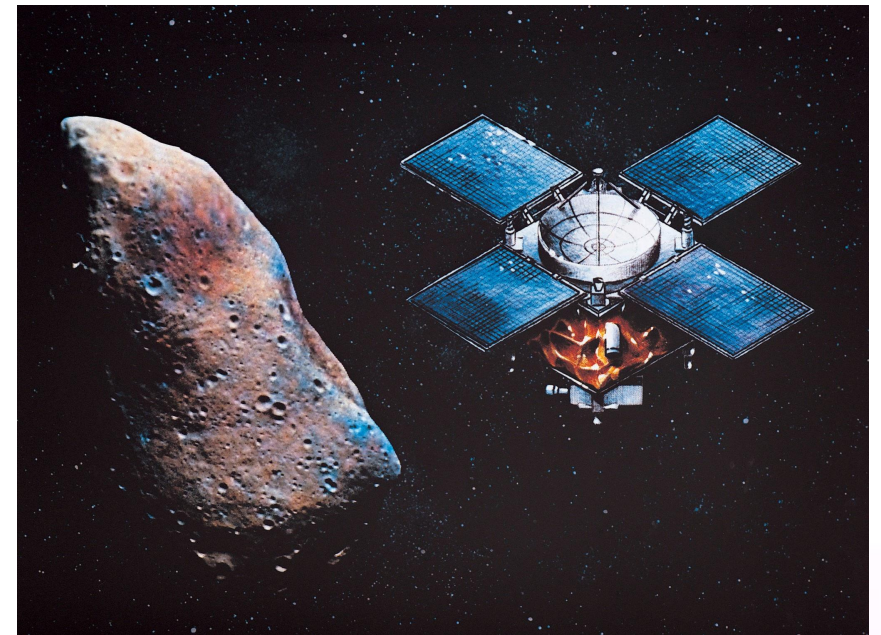
Крок 13: Перевірка повітря

Виведіть значення `atmosphere_safe` на екран через `print()`. Комп'ютер має підтвердити безпеку.»



Крок 14: Наближення метеорита

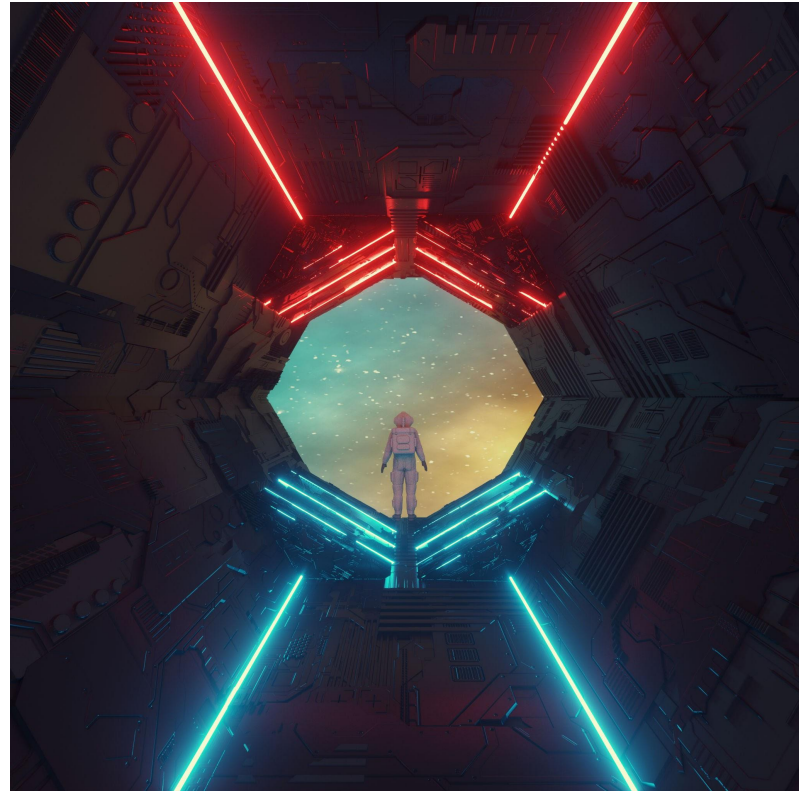
Радари засікли загрозу! Створіть змінну `meteor_distance` (відстань до метеорита) і запишіть туди число **450** (кілометрів).



Крок 15: Стан енергощитів

Яка потужність силового поля?

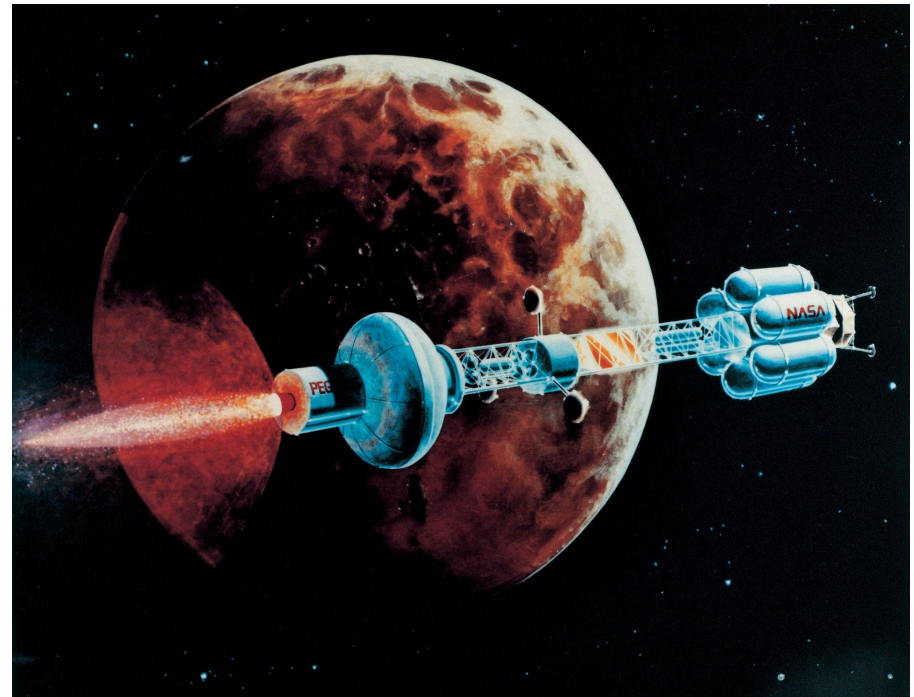
Створіть змінну `shield_power` (енергія щитів) і запишіть туди 40 відсотків.



Крок 16: Логічне «АБО» (or)

Час для Червоної Тривоги!

Створіть змінну `red_alert`. Тривога вмикається, якщо метеорит занадто близько — менше ніж 500 км (`meteor_distance < 500`) АБО (or) щити занадто слабкі — менше ніж 50 відсотків (`shield_power < 50`).



Крок 17: Перевірка сирени

Увімкнулася сирена?

Надрукуйте в консоль статус `red_alert` за допомогою `print()`

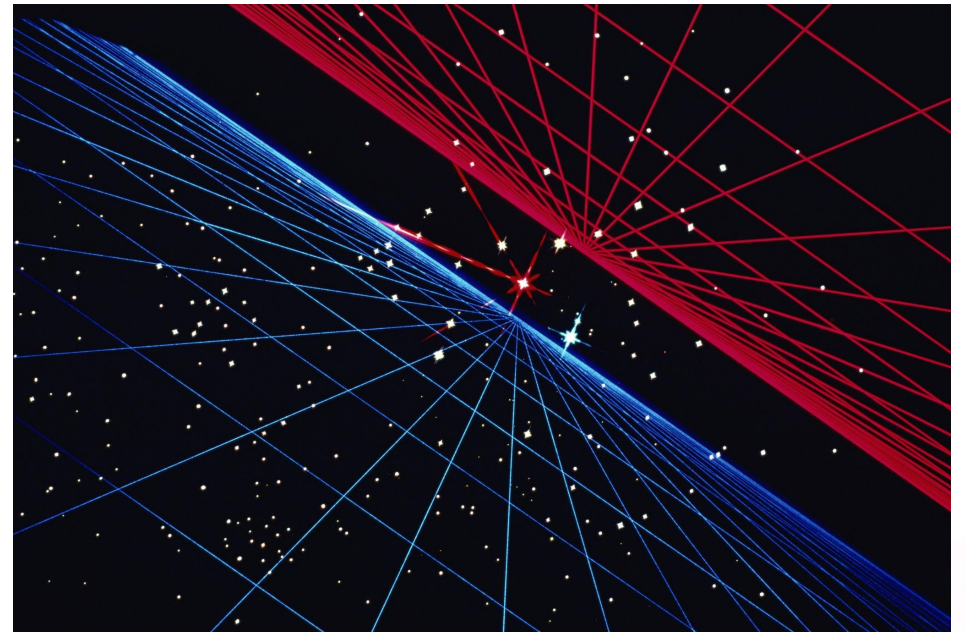


Крок 18: Складна логіка з дужками

Капітан хоче зробити Гіперстрибок, щоб утекли від метеорита.

Створіть змінну `hyperjump_ready`.

Стрибок можливий, лише якщо атмосфера безпечна (`atmosphere_safe`) і (and) виконується одна з умов у дужках: або тривоги немає (`red_alert == False`), або щити заряджені більше ніж на 30 відсотків (`shield_power > 30`)



Крок 19: Фінальний тест систем

Перевіряємо готовність до стрибка!
Виведіть на екран `hyperjump_ready`.
Чи зможемо ми втекти?



Крок 20: Поділ трофеїв (/)

Ми успішно стрибнули і знайшли занедбаний супутник з криптовалютами!

У нас є **100** монет.

Створіть змінну `coins_per_member` і порахуйте, скільки отримає кожен астронавт, поділивши **100** на кількість екіпажу (`crew_members`).

Виведіть результат на екран!



Повний фінальний код

Кроки 1-4: Базові змінні

```
station_name = "Оріон-9"  
crew_members = 4  
food_containers = 15  
portions_per_container = 20
```

Кроки 5-9: Математика їжі

```
total_portions = food_containers * portions_per_container  
portions_per_day = 3  
daily_station_consumption = crew_members * portions_per_day  
days_of_food = total_portions / daily_station_consumption  
print(days_of_food)
```

Кроки 10-13: Логіка атмосфери

```
oxygen_system = True  
air_leak = False  
atmosphere_safe = oxygen_system and air_leak == False  
print(atmosphere_safe)
```

Кроки 14-17: Червона тривога

```
meteor_distance = 450  
shield_power = 40  
red_alert = meteor_distance < 500 or shield_power < 50  
print(red_alert)
```

Кроки 18-19: Готовність до стрибка

```
hyperjump_ready = atmosphere_safe and (red_alert == False or  
shield_power > 30)  
print(hyperjump_ready)
```

Крок 20: Трофеї

```
coins_per_member = 100 / crew_members  
print(coins_per_member)
```